



例题及作业 — Chapter 7 Relational Database Design

2023 年 10-11
月

School of Computer Science



北京邮电大学

关系数据库设计内容 / 步骤

Why? 数据冗余:

1. 1:n 的 2 个 entity "放置"在 1 个模式内

2. 模式内存在 1 个或多个多值属性

不良模式分析

Pitfalls: 数据冗余、插入异常、删除异常、更新异常

范式判断

"好"模式的标准: 1NF, 2NF, 3NF, BCNF, 4NF, 5NF

模式分解的要求

无损连接: 保证信息不丢失、不混淆

减少冗余: 范式的判定

保持函数依赖 (依赖闭包、正则覆盖): 降低依赖 / 约束测试成本

模式分解方法

分解为 BCNF 范式, 无损连接;

分解为 3NF 范式, 无损连接 + 依赖保持

去范式的模式设计

denormalization: BCNF $\rightarrow$ 3NF, 3NF $\rightarrow$ 2NF

目标: 适当的数据冗余, 避免多表查询, 提高查询效率

要求

- 了解由数据依赖导致的不合理的关系模式带来的插入 / 删除 / 更新异常、数据冗余
 - 以 *ins_dept*(*ID*, *name*, *salary*, *dept_name*, *building*, *budget*) in 7.1 为例
 - *inst_info*(*ID*, *child_name*, *phone*) in 7.3.4 Higher Normal Forms
 - 数据依赖
 - 函数依赖
- 掌握函数依赖相关概念 (7.2.2) , 能够分析、证明函数依赖
 - 特殊类型函数依赖
 - 平凡依赖 (trivial, 7.2.2)
 - 部分依赖 (partial, Exercise 7.19)
 - 传递依赖 (transitive, Exercise 7.18)

要求

- - ▣ 判定关系实例 $r(R)$ 满足函数依赖 $\alpha \rightarrow \beta$
 - 根据函数依赖定义, 判断关系实例 $r(R)$ 是否满足函数依赖, 关系实例是否合法/legal
 - ▣ 判定、证明函数依赖是否成立, $\alpha \rightarrow \beta$ holds on $r(R)$
 - 方法1: 按照定义, 使用Armstrong公理系统(7.4.1), 例如, 如果 $A \rightarrow B, B \rightarrow C$, 则 $A \rightarrow C$
 - 方法2: 掌握属性闭包 $(\alpha)^+$ 的概念, 使用属性闭包算法, 判定函数依赖是否成立(7.4.2):
$$\alpha \rightarrow \beta \text{ holds on } R, \text{ if } \beta \subseteq (\alpha)^+$$
- 能够使用SQL语句定义、验证键和函数依赖
 - ▣ 方式: 元组变量, group-by, not exist
 - ▣ 形式: check, assertion, trigger

要求

- 掌握计算关系模式的候选键、判断主属性的方法
 - ▣ 方法 1 : 根据定义, 推断候选键: $\alpha \rightarrow R?$, $(\alpha)^+ = R?$
 - ▣ 方法 2 : 使用补充算法 (附录), 计算关系模式的 (多个) 候选键 (重点)
 - ▣ 主属性 prime attribute, in Exercise 7.18

要求

- 了解多值依赖概念 (7.3.4, 7.6.1) , 以及 4NF 定义 (7.6.2)
 - *inst_info*(*ID*, ***child_name***, ***phone***) in 7.3.4 Higher Normal Forms
 - *inst*(***ID***, ***depart_name***, *name*, *street*, *city*) in 7.6 Decomposition using multivalued dependency
- 理解数据库模式设计范式的含义 , 掌握 1NF 、 2NF 、 3NF 、 BCNF 的定义和性质 , 能够判别关系模式所属范式 , 能够证明包含关系
 - BCNF, 3NF
 - BCNF, in 7.3.1.1, 3NF, in 7.3.2/Exercise 7.18
 - 1NF, 7.8 Atomic Domains and First Normal Form
 - 2NF, in Exercise 7.19

函数依赖保持性

无损连接性

1NF

←-----

消除非主属性对键的部分函数依赖

2NF [特点：不存在非主属性对键的部分依赖]

←-----

消除非主属性对键的传递函数依赖

3NF [特点：不存在非主属性对键的部分、传递依赖]

←-----

消除主属性对键的部分和传递函数依赖

BCNF [不存在非主属性、主属性对键的部分、传递依赖]

←-----

消除非平凡且非函数依赖的多值依赖

4NF

←-----

消除非平凡连接依赖

5NF

Fig. 7.0.3 关系范式特点及间相互关

要求

- 了解函数依赖间的逻辑蕴含、函数依赖闭包 F^+ ，以及 Armstrong 公理系统 (7.2.2,7.4.1)
 - 函数依赖可能蕴含其他依赖，分解之后的模式要保证原有模式之上的所有依赖能够得到快速验证
- 属性闭包 $(\alpha)^+$ 的概念，使用属性闭包算法，判定函数依赖是否成立 (7.4.2)
- 能够化简函数依赖集，计算函数依赖集 F 的正则覆盖 F_c (7.4.2)
 - 冗余函数依赖、冗余属性

要求

- 能够判定模式分解的性质
 - 分解的无损连接(7.1.2)及其判定方法(7.2.3)
 - R分解为 R_1, R_2 , 要求: $R_1 \cap R_2 \rightarrow R_1$, 或 $R_1 \cap R_2 \rightarrow R_2$
，即分解后的子模式 R_1 与 R_2 间存在外键关联
 - 分解的函数依赖保持 (7.3.1.2)及其判定方法(7.4.4)
- 掌握BCNF分解和3NF分解算法及其特点
 - BCNF分解算法(7.5.1.2), 保证无损连接, 不保证函数依赖保持
 - 3NF分解算法(7.5.2), 同时保证无损连接、函数依赖保持

■ I. 函数依赖

- 1. 给定关系表 $r(R)$ 和若干函数依赖 $\alpha \rightarrow \beta$, 判断 r 是否满足 $\alpha \rightarrow \beta$
- 2. 判断函数依赖是否成立
 - 方法1. 按照定义, 以及Amstron公理系统, 判断、证明关于函数依赖的一些公式
 - 方法2. 使用属性闭包算法, 判定函数依赖是否成立(7.4.2):

$$\alpha \rightarrow \beta \text{ holds on } R, \text{ if } \beta \subseteq (\alpha)^+$$

- 3. 根据文字描述, 抽象出函数依赖关系
- 4. **设计SQL语句, 判断函数依赖、键**

■ II. 利用补充算法, 求复杂关系模式的1个、多个候选键

习题类型

- III. 给定关系模式 R 和定义在 R 上的函数依赖集 F ，判断 R 属于第几范式，为什么？
 - 只考虑 BCNF, 3NF, 2NF
 - 2NF: 部分、传递依赖，主属性、非主属性
- IV. 判断一个模式分解的性质
 - 是否为无损连接（定义），判定定理
 - 是否为函数依赖保持
- V. 计算属性闭包 α^+
- VI. 判断冗余属性，计算函数依赖集 F 的最小正则集 F_c
- VII. 给定非 3NF 的关系模式 R 和定义在 R 上的函数依赖集 F ，判别所属最高范式，将 R 分解为 3NF
- VIII. 给定非 BCNF 的关系模式 R 和定义在 R 上的函数依赖集 F ，判别所属最高范式，将 R 分解为 BCNF 范式

Example I-1-1 关系实例满足函数依赖

- Given relation $r(R)$ shown below, which FD is satisfied by r

□ A. $A \rightarrow B$ B. $AC \rightarrow B$ C. $BC \rightarrow A$ D. $B \rightarrow C$

	A	B	C
t1	1	4	2
t2	3	5	6
t3	3	4	6
t4	7	3	8
t5	9	1	0

- $t2[A]=t3[A]=3, t2[B]=5 \neq t3[B]=4$,
 $A \rightarrow B$ is not satisfied
- $t2[AC]=t3[AC]=36, t2[B]=5 \neq t3[B]=4$,
 $AC \rightarrow B$ is not satisfied
- $BC \rightarrow A$ is satisfied
- $t1[B]=t3[B]=4, t1[C]=2 \neq t3[C]=6$,
 $B \rightarrow C$ is not satisfied

Example I-2 函数依赖成立 / 性质

例 1.

■ Question

Which rule about functional dependencies shown below is right

- A. if $\alpha \rightarrow \beta$ then $\beta \rightarrow \alpha$
- B. if $A \rightarrow C, BC \rightarrow D$ then $AB \rightarrow D$
- C. if $AB \rightarrow C$ then $B \rightarrow C$
- D. if $\alpha \subseteq \beta$, then $\alpha \rightarrow \beta$

■ Answer: B

例 2.

■ Exercise 7.4 in the textbook

Example I-3-1 描述函数依赖

- Consider the schema $R = (\text{employee_ID}, \text{date}, \text{turnover per-day}, \text{department_name}, \text{manager})$ that describes the information about the turnover per-day (日营业额) for each employee everyday, the department that each employee works at, and the manager of the department the employee works at.
 - it is assumed that
 - at every *day*, each *employee* has *only one turnover per-day*
 - each *employee* works at *only one department*
 - each *department* is managed by *only one manager*
- According to the descriptions mentioned above, list all the functional dependencies that hold on R
 - $F = \{ \text{employee_ID}, \text{date} \rightarrow \text{turnover per-day},$
 $\text{employee_ID} \rightarrow \text{department_name},$
 $\text{department_name} \rightarrow \text{manager}$
 $\}$
- Q&A: *How about the E-R diagram for this example?*

Example I-4-1 SQL Statements for FD Judgment

- 给定关系模式 $R(A, B, C)$ 和关系 $r(R)$, 利用SQL语句判断关系 $r(R)$ 是否满足函数依赖

$$B \rightarrow C$$

7.9 Given the database schema $R(A, B, C)$, and a relation r on the schema R , write an SQL query to test whether the functional dependency $B \rightarrow C$ holds on relation r . Also write an SQL assertion that enforces the functional dependency. Assume that no null values are present. (Although part of the SQL standard, such assertions are not supported by any database implementation currently.)

- 要求:

如果 $r(R)$ 不满足该函数依赖, 找出导致不满足的 $r(R)$ 中的元组。

- 方法

- 1. 使用元组变量
- 2. group by
- 如果查询结果为null, not exist, 则函数依赖成立

r			s		
A	B	C	A	B	C
1	2	3	1	2	3
1	2	3	1	2	4
1	2	4	1	2	3
1	2	4	1	2	4

```
select *
from r
group by B
having count(*)>1
```

```
select *
from r as T, r as S
where T.B=S.B
and T.C<>S.C
```

Example I-4-2 SQL for FD Judgement

- Use one or more SQL statements to verify whether or not `customerID` is the primary key in the table `customer(customerID, name, address)`, i.e. the functional dependency `customerID → name, address` is satisfied by the table, according to the query results of one or more SQL statements (5 points)
- 方法1 group-by
 - 如果查询结果大于1, 则`customerID`不是主键

```
select  max(count(*))  
from    customer  
group by customerID
```

```
select max(numTuple)  
from {  
    select customerID, count(*) as numTuple  
    from   customer  
    group by customerID  
}
```

Example: SQL for FD Maintenance

- 设计一个断言，以保证在 $R(A, B, C)$ 上成立 $B \rightarrow C$

create assertion B2C check

(not exists

(select B

from R

group by B

having count(distinct c) > 1

)

)

Example I-4-2 : SQL for FD Judgement

- 方法 2 元组变量

```
select *  
from customer as A, customer as B  
where A.customerID = B.customerID  
      and (A.name<>B.name OR A.address<>B.address)
```

- 如果该语句查询结果为空，则 *customerID* 是主键

Appendix II-0 Algorithm for Computing Candidate Keys

- 定理
 - ▣ $R, F=\{\alpha \rightarrow \beta\}$ holds on R
 - ▣ 如果属性 A 只出现在 F 中函数依赖的左部 α ，则 A 一定是主属性，必然出现在候选键中
- 输入：关系模式 R 及其函数依赖集 F
- 输出： R 的所有候选关键字
- 方法：
 - step1. 将 R 的所有属性分为四类：
 - L 类：仅出现在 F 中函数依赖左部的属性
 - R 类：仅出现在 F 中函数依赖右部的属性
 - N 类：在 F 中函数依赖左右两边均未出现的属性
 - LR 类：在 F 中函数依赖左右两边均出现的属性

Computing Candidate Keys

- 令 $X_set = L类 \cup N类$, $Y_set = LR类$;
- step2. 求 X_set^+ ,
如果 $X_set^+ = R$, 则 X_set 为R的唯一候选键, 转step5;
否则转step3
- step3. 在 Y_set 中取一属性A, 求 $(X_set \cup A)^+$, 若它包含了R的所有属性, 则转step4;
否则, 调换一属性反复进行这一过程, 直到试完所有 Y_set 中的属性
- step4. 如果已找出所有的候选键, 则转step5;
否则, 在 Y_set 中依此取两个、三个, ..., 求其属性闭包, 直至其闭包包含R的所有属性
- step5. 停止, 输出结果

Example II-1 计算候选键

- $R(X, Y, Z, W)$, and

$$F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$$

- L 类 : Y, W
- R、N: 空
- LR: X, Z
- $X_set = \{Y, W\}$
- $Y_set = \{X, Z\}$
- $(YW)^+ = R$, so YW is the unique candidate key

- $R(B, D, I, O, Q, S)$, and

$$F = \{B \rightarrow Q, I \rightarrow S, IS \rightarrow Q, S \rightarrow D\}$$

- L 类 : B, I
- R 类 : D, Q
- N 类 : O
- LR 类 : S
- $X_set = \{B, I, O\}$
- $Y_set = \{S\}$
- $(BIO)^+ = R$, so BIO is the unique candidate key

Example II-2 计算候选键

- $R(A, B, C, D, E)$
- $F=\{AB \rightarrow CE, E \rightarrow AB, C \rightarrow D\}$
- Step1. $X_set = \text{空}$, $Y_set = \{A, B, C, E\}$
 - L 类 : 空
 - R 类 : D
 - LR 类 : A, B, C, E
 - N 类 : 空
- Step2. 从 $Y_set = \{A, B, C, E\}$ 中只取 1 个属性 , A, B, C, E ,
 - $(A)^+ \neq R, (B)^+ \neq R, (C)^+ \neq R$, A 、 B 、 C 不是候选键
 - $E \rightarrow R$, E 是一个候选键
 - 且其它任何包含 E 的属性集不可能是候选键
- Step3. 从 Y_set 中取 2 个属性 (不包括 E) , AB, AB $\rightarrow R$; BC, $(BC)^+ \neq R$,
 - AB 是一个候选键 , 且其它任何包含 AB 的属性集不可能是候选键
- Step4. 无法从 Y_set 中取 3 个属性 , 使得这 3 个属性不包括 E 、 不包括 AB
- 因此 , 只有 2 个候选键 E, AB

Example II-3 计算候选键

- 给定关系模式 $R(A,B,C,D,E)$ ，以及函数依赖集合 $A \rightarrow BC, CD \rightarrow E, B \rightarrow D, E \rightarrow AB$ ，找出 R 的

全部候选键

- 答案【4 个候选键】

- 出现在左边的属性 $\{A,C,D,B,E\}$ ，出现在右边的属性 $\{A,C,D,B,E\}$
- L 类属性 $\{\}$
- R 属性 $\{\}$
- LR 类双部属性 $\{A,C,D,B,E\}$

- 计算候选键

- 测试 $A^+ = \{A,B,C,D,E\}$, A 是候选键，包含 A 的其他所有属性集合都不是候选键
- 测试 $E^+ = \{A,B,C,D,E\}$, E 是候选键，包含 E 的其他所有属性集合都不是候选键
- 测试 B^+, C^+, D^+ ，都不能包含所有的属性
- 测试 $CD^+ = \{A,B,C,D,E\}$, CD 是候选键，包含 CD 的其他所有属性集合都不是候选键
- 测试 BD^+ 都不能包含所有的属性；
- 测试 $BC^+ = \{A,B,C,D,E\}$, BC 是候选键，包含 BC 的其他所有属性集合都不是候选键

Example III-1-1 BCNF 判別

- Example schema that is **not** in BCNF:
in_dep (ID, *name*, *salary*, *dept_name*, *building*, *budget*)
because :
 - ▣ *dept_name* → *building*, *budget*
 - holds on *in_dep*
 - but
dept_name is not a superkey
- When decompose *in_dept* into *instructor* and *department*
 - ▣ *instructor* is in BCNF
 - ▣ *department* is in BCNF

Example III-2-1 3NF 判別

- Consider a schema:

dept_advisor(s_ID, i_ID, dept_name)

- With function dependencies:

$i_ID \rightarrow dept_name$

$s_ID, dept_name \rightarrow i_ID$

- Two candidate keys = $\{s_ID, dept_name\}, \{s_ID, i_ID\}$

- We have seen before that *dept_advisor* is **not** in BCNF

- *R*, however, is in 3NF

- ▣ $s_ID, dept_name$ is a superkey
- ▣ $i_ID \rightarrow dept_name$ and i_ID is NOT a superkey, but:
 - $\{dept_name\} - \{i_ID\} = \{dept_name\}$ and
 - $dept_name$ is contained in a candidate key

Example III-3-1 2NF 判别

- E.g. SLC(S# , SDpt, SLocation, C# , Grade)
 - $(S\#, C\#) \rightarrow G$, $SDpt \rightarrow SLocation$,
 $S\# \rightarrow SDpt$, $(S\#, C\#) \rightarrow SDpt$,
 $S\# \rightarrow SLocation$, $(S\#, C\#) \rightarrow SLocation$
- SLC is not in 2NF, because
 - for non-primary attribute *SDpt*, there exists $S\# \rightarrow SDpt$, so *SD* is partially dependent on the key $(S\#, C\#)$
 - for non-primary attribute *SL*, there exists $S\# \rightarrow SLocation$, so *SLocation* is partially dependent on the key $(S\#, C\#)$
- SLC is decomposed into two 2NF schemas
 - $(S\#, C\#, Grade)$
 - $(S\#, SDpt, SLocation)$
- E.g. R(Sno , Sname, Status, City, Pno , Qty)
 - $Sno \rightarrow Sname$, $Sno \rightarrow Status$,
 $Status \rightarrow City$,
 $(Sno, Pno) \rightarrow Qty$
- R is not in 2NF, because
 - for non-primary attributes *Sname* and *Status* are partially dependent on the key (Sno, Pno)
- R is decomposed into two 2NF schemas
 - $R_1(\underline{Sno} , Sname, Status, City)$
 - $R_2(Sno, Pno, Qty)$

Example IV-1 Lossless Decomposition/ Dependency Preserving Judgement

- $R = (A, B, C)$
 $F = \{A \rightarrow B, B \rightarrow C\}$
- $R_1 = (A, B), R_2 = (B, C)$
 - Lossless-join decomposition:
 $R_1 \cap R_2 = \{B\}$ and $B \rightarrow BC$
 - Dependency preserving
- $R_1 = (A, B), R_2 = (A, C)$
 - Lossless-join decomposition:
 $R_1 \cap R_2 = \{A\}$ and $A \rightarrow AB$
 - Not dependency preserving
(cannot check $B \rightarrow C$ without computing $R_1 \bowtie R_2$)

Example V-1 Attribute Set Closure

- $R = (A, B, C, G, H, I)$
- $F = \{A \rightarrow B$
 $A \rightarrow C$
 $CG \rightarrow H$
 $CG \rightarrow I$
 $B \rightarrow H\}$
- $(AG)^+$
 - ▣ 1. *result* = **AG**
 - ▣ 2. *result* = **ABCG** ($A \rightarrow C$ and $A \rightarrow B$)
 - ▣ 3. *result* = **ABCGH** ($CG \rightarrow H$ and $CG \subseteq AGBC$)
 - ▣ 4. *result* = **ABCGHI**
($CG \rightarrow I$ and $CG \subseteq AGBCH$)
- $(AG)^+ = R$, AG is a *superkey* of R
- Is AG a candidate key?
 - ▣ 1. Is AG a super key?
 1. Does $AG \rightarrow R$? == Is $R \supseteq (AG)^+$
 - ▣ Is any subset of AG a superkey?
 1. Does $A \rightarrow R$? == Is $R \supseteq (A)^+$, *no*
 2. Does $G \rightarrow R$? == Is $R \supseteq (G)^+$, *no*
 3. In general: check for each subset of size $n-1$

Example VI-1 Extraneous Attributes

- Example: Given $F = \{A \rightarrow C, AB \rightarrow C\}$
 - ▣ B is extraneous in $AB \rightarrow C$ because $\{A \rightarrow C, AB \rightarrow C\}$ logically implies $A \rightarrow C$ (i.e. the result of dropping B from $AB \rightarrow C$).
- Example: Given $F = \{A \rightarrow C, AB \rightarrow CD\}$
 - ▣ C is extraneous in $AB \rightarrow CD$ since $AB \rightarrow C$ can be inferred even after deleting C
- Let $F = \{AB \rightarrow CD, A \rightarrow E, E \rightarrow C\}$
- To check if C is extraneous in $AB \rightarrow CD$, we:
 - ▣ Compute the attribute closure of AB under $F' = \{AB \rightarrow D, A \rightarrow E, E \rightarrow C\}$
 - ▣ The closure is $ABCDE$, which includes CD
 - ▣ This implies that C is extraneous

Example VII-1 3NF 分解

- Consider schema $R(X, Y, Z, W)$, and $F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$ that holds on R . Give the lossless, dependency-preserving decomposition of this schema into 3NF

Answer:

- Step1. $\{YW\}^+ = R$, $\{YW\}$ is the unique candidate key
- Step2. With respect to $F = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\}$,
 - considering X in $Y \rightarrow XZ$ and Z in $W \rightarrow XZ$,
 $\{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$ implies F , so X in $Y \rightarrow XZ$ and Z in $W \rightarrow XZ$ are all extraneous attributes, and one of **canonical covers** for F is

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$$

使用补充算法计算候选键：

$$L = \{Y, W\}$$

$$R = \{X, Z\}$$

$$\begin{aligned} F &= \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow XZ, W \rightarrow XZ\} \\ &= \{ \underline{X} \rightarrow Z, Z \rightarrow X, \\ &\quad \underline{Y} \rightarrow X, Y \rightarrow Z, \\ &\quad W \rightarrow X, \underline{W} \rightarrow Z \} \end{aligned}$$

Example VII-1 3NF 分解

- the other *canonical covers* for F are

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow Z\}, \text{ or}$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow X\}, \text{ or}$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow Z\}$$

- Step3. As far as $F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow X\}$ is concerned,

- on the basis of 3NF decomposition algorithm, the subschema responding to each functional dependency in F_c are

$$R_1 = \{XZ\}, R_2 = \{YZ\}, R_3 = \{XW\}$$

- for the candidate key $\{YW\}$, $R_4 = \{YW\}$ is generated, so *one of* the 3NF decomposition is

$$\{XZ, YZ, XW, YW\}$$

Example VII-1 3NF 分解

- With respect to the other *canonical covers*

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow X\}, \text{ or}$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow X, W \rightarrow Z\}, \text{ or}$$

$$F_c = \{X \rightarrow Z, Z \rightarrow X, Y \rightarrow Z, W \rightarrow Z\}$$

and the candidate key $\{YW\}$,

the corresponding 3NF decomposition are

$$\{XZ, YX, WX, YW\}$$

$$\{XZ, YX, WZ, YW\}$$

$$\{XZ, YZ, WZ, YW\}$$

Example VII-3-2 范式判别及 3NF 分解

- Consider the following relation schema R about student information and the functional dependency F hold on R:
 - ▣ $R(stNo, Name, Telno, stId, Classno, Cno, Cname, Grade)$
 - ▣ $F = \{stNo \rightarrow Name, Telno, stId, Classno, stId \rightarrow stNo, stNo, Cno \rightarrow Grade, Cno \rightarrow Cname\}$
- List all the candidate keys of R
 - ▣ $(stNo, Cno) (stId, Cno)$
- What is the highest normal form of R? Why?
 - ▣ not in BCNF, because of FD 1, 2, and 4
 - ▣ not in 3NF, because for FD 1, each attribute in $\beta - \alpha = \{Name, Telno, Classno\}$ is **not** contained in candidates
 - ▣ not in 2NF  , because in FD1, **non-prime attribute** is partially dependent on candidates

使用补充算法计算候选键：

$L = \{Cno\}$

$LR = \{stNo, stId\}$

$R = \{Name, Telno, Classno, Cname, Grade\}$

$(Cno, stNo)^+ = R$

$(Cno, stId)^+ = R$

Example VII-3-2 范式判别及 3NF 分解

- Is the decomposition $\rho = \{R_1(stNo, Name, Telno, stId, Classno, R_2(stId, Cno, Cname, Grade) \}$ on R lossless-join? Why?
 - ▣ is a lossless-join decomposition, because $R_1 \cap R_2 = Id \rightarrow R_1$
 - Give a lossless-join and dependency-preserving decomposition of R into 3NF
 - ▣ $F_c = F$
 - ▣ 3NF decomposition
 - $R_1(stNo, Name, Telno, stId, Classno)$
 - $R_2(stNo, Cno, Grade)$
 - $R_3(Cno, Cname)$
 - $R_4(stID, Cno)$
- $F =$

 - $\{stNo \rightarrow Name, Telno, stId, Classno,$
 - $stId \rightarrow stNo,$
 - $stNo, Cno \rightarrow Grade,$
 - $Cno \rightarrow Cname \quad \}$

Candidate keys:

 - $(stNo, Cno)$
 - $(stId, Cno)$
-

Example VII-3-3 范式判别及 3NF 分解

- Consider the relation schema Sales about saling information and the functional dependency F hold on Sales:
 - ▣ Sales(Sales-TransactionNo, Item-No, Item-Price, Item-QuantitySold, Seller, SellerDistrict)
 - ▣ $F = \{ \text{Sales-TransactionNo, Item-No} \rightarrow \text{Item-QuantitySold}$
 $\text{Item-No} \rightarrow \text{Item-Price}$
 $\text{Sales-TransactionNo} \rightarrow \text{Seller}$
 $\text{Seller} \rightarrow \text{SellerDistrict} \}$
- The candidate key: Sales-TransactionNo, Item-No
- Sales is in 1NF, because for Item-No $\rightarrow$ Item-Price, the non-prime attribute Item-Price is partially dependent on the candidate key
 - ▣ 存在非主属性 item-Price 对候选键 {Sales-TransactionNo, Item-No} 的部分依赖，不满足 2NF 要求
- 3NF decomposition
 - ▣ $F = F_c$
 - ▣ {Sales-TransactionNo, Item-No, Item-QuantitySold}, {Item-No, Item-Price}, {Sales-TransactionNo, Seller}, {Seller $\rightarrow$ SellerDistrict}

Example VII-3-4 范式判别及 3NF 分解

- Considering the schema $R=(\text{athlete-id}, \text{game-event}, \text{grade}, \text{category of games}, \text{manger of games})$ that describes the sports meeting. It is assumed that
 - ▣ for each athlete, if he takes part in a game event, he will achieve one and only one grade
 - ▣ each game event belongs to one and only one category
 - ▣ each category is managed by one and only one manager
- According to the descriptions mentioned above, list the functional dependency set F that holds on R
 - ▣ List all the candidate keys of R .
 - ▣ Give a lossless and dependency-preserving decomposition of R into 3NF

Example VII-3-4 范式判别及 3NF 分解

- 函数依赖如下：
 - ▣ athlete-id, game-event → grade
 - ▣ game-event → category of games
 - ▣ category of games → manger of games
- 候选键是：
 - ▣ athlete-id, game-event
- 分解的 3NF 是：
 - ▣ (athlete-id, game-event, grade)
 - (game-event → category of games)
 - (category of games, manger of games)

Example VII-3-5 范式判别及 3NF 分解

- Consider the following relation schema R about student information and the functional dependency F hold on R:
 - $R(A, B, C, D, E)$
 - $F = \{A \rightarrow C, C \rightarrow A, B \rightarrow A, D \rightarrow AC, B \rightarrow E\} = \{A \rightarrow C, C \rightarrow A, B \rightarrow A, B \rightarrow E, D \rightarrow A, D \rightarrow C\}$
- List all the candidate keys of R
 - an unique candidate key **BD**
- What is the highest normal form of R, and why?
 - 对 $A \rightarrow C$ ，左部 A 不是候选键，同时也没有包括在候选键中——不属于主属性，因此 R 不是第 3 范式
 - $B \rightarrow A$ means the non-prime attribute A is partially dependent on **BD**, so R is not in 2NF
 - 存在非主属性 A 对候选键 BD 的部分依赖，故不满足第 2 范式的定义
 - **So, R is in 1NF**

使用补充算法计算候选键：

$L = \{B, D\}$

$LR = \{A, C\}$

$R = \{E\}$

Example VII-3-5 范式判别及 3NF 分解

- Compute the canonical cover F_c

- $F_c = \{A \rightarrow C, C \rightarrow A, B \rightarrow AE, D \rightarrow A\},$

- $F_c = \{A \rightarrow C, C \rightarrow A, B \rightarrow AE, D \rightarrow C\}$

- 方法：

Step1. 将 $F = \{A \rightarrow C, C \rightarrow A, B \rightarrow A, D \rightarrow AC, B \rightarrow E\}$ 变换为

$$F = \{A \rightarrow C, C \rightarrow A, B \rightarrow A, D \rightarrow A, D \rightarrow C, B \rightarrow E\},$$

去除其中的传递函数依赖 $D \rightarrow C$ ($D \rightarrow A, A \rightarrow C$) 或 $D \rightarrow A$ ($D \rightarrow C, C \rightarrow A$)

Step2. 合并 $B \rightarrow A, B \rightarrow E$

- Give a lossless and dependency-preserving decomposition of R into 3NF

- for $F_c = \{A \rightarrow C, C \rightarrow A, B \rightarrow AE, D \rightarrow A\},$ a 3NF decomposition is

$$\{AC, ABE, AD, BD\}$$

Example VII-3-6 范式判别及 3NF 分解

- Consider the following relations and answer the Questions
 - the relation schema $R=(A,B,C,D,E,F)$, the functional dependencies set
$$F = \{A \rightarrow B, C \rightarrow D, D \rightarrow E, E \rightarrow F, F \rightarrow C\}$$
 - List all the candidate keys of R
 - AC, AD, AE, AF
- 使用补充算法计算候选键：

$L=\{A\}$
 $LR=\{C,D,E,F\}$
 $R=\{B\}$
- What is the highest normal form of R? Why?
 - R is in 1NF, because there exists $A \rightarrow B$, and the non-prime attribute B is partially dependent on the candidate key, e.g. AC
 - Give a lossless-join and dependency-preserving decomposition of R into 3NF
 - $F_c = F$
 - 3NF decomposition
$$AB, CD, CF, DE, EF, AC, AD, AE, AF$$

Example VII-3-7 范式判别及 3NF/BCNF 分解

- Consider the schema $R=(A, B, C, D, E)$, and $F = \{ AB \rightarrow CE, E \rightarrow AB, C \rightarrow D \}$ that holds on R .
 - List all the candidate keys of R .
 - What is the highest normal form of R ? and Why?
 - Give a lossless-join, dependency-preserving decomposition into 3NF of R
 - Give a lossless-join decomposition into BCNF of R

- Candidate keys

使用补充算法计算候选键：

$L = \{ \}$ $LR = \{A, B, C, E\}$
 $R = \{D\}$

- 由观察得到： $(AB)^+ = E^+ = ABCDE$

- 范式判断

- 由 $C \rightarrow D$ ， C 不是候选键，故 R 不是 BCNF
- 对 $C \rightarrow D$ ，右端 D 不是任一候选键的一部分，即不是主属性，故 R 不是 3NF
- A 、 B 、 E 是主属性，非主属性 C 、 D 没有部分依赖于候选键 AB 和 E ，因此 R 最高为 2NF

Example VII-3-7 范式判别及 3NF 分解

■ 3NF 分解

▣ 先计算 F_c

- 各 FD 左部都不同，不需合并
- 对于 $AB \rightarrow CE$ ， $E \rightarrow AB$ ， $C \rightarrow D$ 均无冗余属性
- 因此， $F_c = F = \{AB \rightarrow CE, E \rightarrow AB, C \rightarrow D\}$

▣ 分解：

- 由 $AB \rightarrow CE$ ， $E \rightarrow AB$ 构造 $R_1 = (ABCE)$
- 由 $C \rightarrow D$ 构造 $R_2 = (CD)$
- R_1 种包含候选键， R_1 和 R_2 即为所求分解

■ BCNF 分解

▣ $C \rightarrow D$ 导致 R 不是 BCNF

▣ 由 $C \rightarrow D$ 将 R 分解为

- $R_1 = (ABCE)$ ， $R_2 = (CD)$

▣ $R_1 = (ABCE)$ ， $F_1 = \{AB \rightarrow CE, E \rightarrow AB\}$

$R_2 = (CD)$ ， $F_2 = \{C \rightarrow D\}$

均为 BCNF

Example VIII-1 BCNF 分解

- class (*course_id*, title, dept_name, credits,
sec_id, semester, year, building, room_number, capacity,
time_slot_id) course + classroom + section ▶
- Functional dependencies:
 - $course_id \rightarrow title, dept_name, credits$
 - $building, room_number \rightarrow capacity$
 - $course_id, sec_id, semester, year \rightarrow building, room_number, time_slot_id$
- A candidate key
 $\{course_id, sec_id, semester, year\}$.

使用补充算法计算候选键：

$L = \{course_id, sec_id, semester, year\}$

$R = \{title, dept_name, credits, capacity, time_slot_id\}$

$LR = \{building, room_number\}$

$L^+ = R$

Example VIII-1 BCNF 分解

- BCNF Decomposition:
 - $course_id \rightarrow title, dept_name, credits$ holds
 - but $course_id$ is not a superkey.
 - We replace $class$ by:
 - $course(course_id, title, dept_name, credits)$
 - $class-1(course_id, sec_id, semester, year, building, room_number, capacity, time_slot_id)$
- $course$ is in BCNF
 - How do we know this?
with $F_{course} = \{course_id \rightarrow title, dept_name, credits\}$ holds on $course$,
and $course_id$ is the candidate key !

Example VIII-1 BCNF 分解

- *class-1* (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *capacity*, *time_slot_id*)
 - $F_{\text{class-1}} = \{ \text{building, room_number} \rightarrow \text{capacity} \mid \text{course_id, sec_id, semester, year} \rightarrow \text{building, room_number, time_slot_id} \}$
 - candidate key {*course_id*, *sec_id*, *semester*, *year*}

- *class-1* is not in BCNF

- *building, room_number* $\rightarrow$ *capacity* holds on *class-1*, but {*building, room_number*} is not a superkey
- We replace *class-1* by:
 - ***classroom*** (*building*, *room_number*, *capacity*)
 - ***section*** (*course_id*, *sec_id*, *semester*, *year*, *building*, *room_number*, *time_slot_id*)

- ***classroom*** and ***section*** are in BCNF

- ***class*** is decomposed into ***course***, ***classroom***, and ***section***

使用补充算法计算候选键：

$L = \{ \text{course_id, sec_id, semester, year} \}$

$R = \{ \text{capacity, time_slot_id} \}$

$LR = \{ \text{building, room_number} \}$

$L^+ = R$

$F_{\text{classroom}} = \{ \text{building, room_number} \rightarrow \text{capacity} \}$

$F_{\text{section}} = \{ \text{course_id, sec_id, semester, year} \rightarrow \text{building, room_number, time_slot_id} \}$

Example VIII-2 BCNF 分解

- Considering the schema $R(C, T, H, R, S, G)$, and $F=\{CS \rightarrow G, C \rightarrow T, TH \rightarrow R, HR \rightarrow C, HS \rightarrow R\}$, give a decomposition of R into BCNF
- Step1. With respect to R and F , the unique candidate key is HS , and R is not in BCNF
- Step2. Initially, $result:=\{R\}=\{CTHRSG\}$
 - ▣ R is not in BCNF, because there is $CS \rightarrow G$ in F , and CS is not the candidate key of R
 - ▣ $R(CTHRSG)$ is decomposed into $R_1(CSG)$ and $R_2(CTHRS)$
 - ▣ $R_1(CSG)$ is in BCNF, $F_1=\{CS \rightarrow G\}$

Example VIII-2 BCNF 分解

■ Step3. With respect to $R_2(CTHRS)$

- !the restriction of F to $R_2(CTHRS)$ is

$$F_2 = \{C \rightarrow T, TH \rightarrow R, HR \rightarrow C, HS \rightarrow R\}$$

- !the candidate key is HS

- $R_2(CTHRS)$ is not in BCNF, because there is $C \rightarrow T$ in F_2 , and C is not the candidate key of R_2

- $R_2(CTHRS)$ is decomposed into $R_{21}(CT)$ and $R_{22}(CHRS)$

- $R_{21}(CT)$ is in BCNF, $F_{21} = \{C \rightarrow T\}$

Example VIII-2 BCNF 分解

- Step4. With respect to $R_{22}(\text{CHRS})$
 - !!the restriction of F to $R_{22}(\text{CHRS})$ is
 $F_{22} = \{HR \rightarrow C, HS \rightarrow R, CH \rightarrow R\} / * CH \rightarrow TH, TH \rightarrow R$
 - !!the candidate key is HS
 - $R_{22}(\text{CHRS})$ is not in BCNF, because there is $HR \rightarrow C$ in F_{22} , and HR is not candidate key of R_{22}
 - $R_{22}(C\text{HRS})$ is decomposed into $R_{221}(HRC)$ and $R_{222}(\text{HRS})$
 - $R_{221}(HRC)$ is in BCNF
- Step5. With respect to $R_{222}(\text{HRS})$,
 - the restriction of F to $R_{222}(\text{HRS})$ is
 $F_{222} = \{HS \rightarrow R\}$, and the candidate key is HS
 - $R_{222}(\text{HRS})$ is in BCNF
- Finally, the BCNF decomposition of $R(C, T, H, R, S, G)$ is
 - $R_1(CSG)$, $R_{21}(CT)$, $R_{221}(HRC)$, $R_{222}(\text{HRS})$

BCNF分解例2

- 给定关系 $R(A, B, C, D, E)$ ，函数依赖的集合 $F = \{A \rightarrow B, BC \rightarrow D\}$
- 判定表不是BCNF范式，因为 $A \rightarrow B$, A 不是码
- 分解 $R_1 = (A, B)$, $R_2 = (A, C, D, E)$
- R_2 满足BCNF吗?

下一个动画

BCNF分解例2

- 给定关系 $R(A, B, C, D, E)$ ，函数依赖的集合 $F = \{A \rightarrow B, BC \rightarrow D\}$
- 判定表不是BCNF范式，因为 $A \rightarrow B$, A 不是码
- 分解 $R_1 = (A, B)$, $R_2 = (A, C, D, E)$
- R_2 满足BCNF吗?
- $(AC)_F^+ = ACBD$ ，在 R_2 中，存在 $AC \rightarrow D$ ，但是 AC 无法决定 R_2 的所有属性
- 继续分解， $R_3 = (ACD)$, $R_4 = (ACE)$

无备注。

作业 1、2

- 作业 1: Exercise 7.30

a, b, c, d, e

要求：利用 ppt 讲义附录算法 7-2 Computing Candidate Keys ，计算候选键；

- 作业 2 ： Exercise 7.33

作业 3

- Consider the following relation schema R and the functional dependency F hold on $R=\{X, Y, Z, W, Q\}$,
 $F=\{XY \rightarrow ZQ, Q \rightarrow XY, Z \rightarrow W, Q \rightarrow Z\}$
 - ▣ (1) Compute $(XZ)^+$
 - ▣ (2) Is the decomposition $\rho=\{R1(X,Y, Z), R2(Z,W,Q) \}$ on R lossless-join? Why?
 - ▣ (3) What is the highest normal form of R ? Why?

作业 4

- The functional dependency set $F=\{A\rightarrow C, C\rightarrow A, B\rightarrow A, D\rightarrow AC, B\rightarrow E\}$ holds on the relation schema $R = (A, B, C, D, E)$,
 - ▣ (1) Compute $(AB)^+$
 - ▣ (2) List all the candidate keys of R
 - ▣ (3) What is the highest normal form of R , and why?

作业 3 答案

- (1) $(XZ)^+ = XWZ$
- (2) 不是无损连接分解，因为 R1 和 R2 的公共属性为 Z，不能函数确定 R1 或 R2
- (3) 最高第 2 范式

R 的候选键是 XY 和 Q，非键 / 主属性为 W、Z，不存在非键 / 主属性对候选键的部分依赖，W、Z 完全依赖于 Q，符合第 2 范式定义。

但存在非键属性 W 传递依赖于候选键 Q，即 $Q \rightarrow Z$ ， $Z \rightarrow W$

或： $Z \rightarrow W$ 的左端既不是候选键，也没有包含在主键中，所以不是第 3 范式

作业 3 答案

- (4) 最小函数依赖集是：

$$F_c = \{XY \rightarrow ZQ, Q \rightarrow XY, Z \rightarrow W\}$$

对 $F = \{XY \rightarrow ZQ, Q \rightarrow XY, Z \rightarrow W, Q \rightarrow Z\}$ ，首先有 $Q \rightarrow XY$ 。根据 $XY \rightarrow ZQ$ ，有 $XY \rightarrow Z$ ，因此有 $Q \rightarrow XY, XY \rightarrow Z$ ，所以 F 中的传递函数依赖 $Q \rightarrow Z$ 可去掉，得到 F_c 。

剩余的 3 个函数依赖均无冗余属性。

- 根据 F_c ，得到 2 个子模式：

$$R1(X, Y, Z, Q), R2(Z, W),$$

其中子模式 $R1$ 包含了候选键 XY 、 Q ，因此 $R1$ 、 $R2$ 是 R 的一个 3NF 分解。

作业 4 答案

- (1) $(AB)^+ = ACE$
 - (2) 唯一候选键 BD
 - (3) 最高范式为 1NF
 - ▣ 对非平凡函数依赖 $A \rightarrow C$ ，左部不是超键，因此 R 不是 BCNF
 - ▣ 对 $A \rightarrow C$ ，左部 A 不是超键，且右部的 C 没有包括在主键中（不是主属性），因此 R 不是 3NF
 - ▣ 对 $B \rightarrow A$ ，非主属性 A 部分依赖于候选键 BD，故不满足第 2 范式定义，因此 R 不是 2NF
- 或者：对 $B \rightarrow E$ ，非主属性 E 部分依赖于键 BD，
对 $D \rightarrow C$ ，非主属性 C 部分依赖于键 BD

作业 4 答案

- (4) Compute the canonical cover F_c

$$F_c = \{A \rightarrow C, C \rightarrow A, B \rightarrow AE, D \rightarrow A\}$$

或者： $F_c = \{A \rightarrow C, C \rightarrow A, B \rightarrow AE, D \rightarrow C\}$

方法：

Step1. 将 $F = \{A \rightarrow C, C \rightarrow A, B \rightarrow A, D \rightarrow AC, B \rightarrow E\}$ 变换为

$F = \{A \rightarrow C, C \rightarrow A, B \rightarrow A, D \rightarrow A, D \rightarrow C, B \rightarrow E\}$ ，去除其中的传递函数依赖 $D \rightarrow C$ ($D \rightarrow A, A \rightarrow C$)
或 $D \rightarrow A$ ($D \rightarrow C, C \rightarrow A$)

Step2. 合并 $B \rightarrow A, B \rightarrow E$

作业 4 答案

- (5) Give a lossless and dependency-preserving decomposition of R into 3NF

分解 1 : 根据 $F_c = \{A \rightarrow C, C \rightarrow A, B \rightarrow AE, D \rightarrow A\}$, 得到

$R_1(AC), R_2(ABE), R_3(AD)$,

由候选键 BD , 得到 $R_4(BD)$

分解 2 : 根据 $F_c = \{A \rightarrow C, C \rightarrow A, B \rightarrow AE, D \rightarrow C\}$, 得到

$R_1(AC), R_2(ABE), R_3(CD)$,

由候选键 BD , 得到 $R_4(BD)$



Thanks for your
attention